

6.2: Parallelization in Deep Learning Systems

Lecturer: Hao Zhang

Scribe: Ziang Li, Lucas Yan, Yusen Huang, Changli Liu, Hongrui Zhu, Shu Wang, Zach Liu, Kefan Zhang, Zhirui Xia

1 Overview

This lecture studies why parallelization is central to modern machine learning systems and how to reason about parallel execution from the point of view of the computational graph. Earlier parts of the course considered dataflow graphs, automatic differentiation, graph optimization, operator optimization, and run-time scheduling. Parallelization connects all of these topics: once a model and its training computation are represented as a graph, the systems problem is to map that graph onto a cluster of devices while respecting memory capacity, network bandwidth, and dependency constraints.

The lecture has four main goals. First, it explains why single-device execution is no longer enough for frontier machine learning workloads. Second, it revisits the traditional distinction between data parallelism and model parallelism, then refines it into the more general distinction between inter-operator and intra-operator parallelism. Third, it introduces model FLOPs utilization (MFU) as a macro-level measure of parallel efficiency. Finally, it reviews point-to-point and collective communication primitives, which are the communication building blocks used by distributed training systems.

2 Why Parallelization Is Necessary

The motivation for parallelization comes from a widening gap between machine learning demand and hardware supply. Historically, Moore's law described a rapid increase in transistor density, and together with architectural improvements it contributed to steady growth in available compute, on the order of a 2x improvement every 18 months. That growth has since slowed sharply. Recent general-purpose processors improve by only about 1.05x over the same 18-month window, so the supply side has nearly flattened. The compute requirements of major machine learning workloads have moved in the opposite direction, growing roughly 10x every 18 months [1].

This pressure is particularly visible in large neural networks. Empirically, larger models and larger training runs have often produced better accuracy or better downstream performance. For example, the lecture points to the scaling trend behind large language models such as GPT-3, where increasing parameter count, data size, and training compute enabled stronger few-shot performance [2]. The exact model family is less important than the systems lesson: model quality has become tied to training runs whose computation and memory demands exceed what a single accelerator can provide.

Specialized hardware helps but does not remove the problem. GPUs and TPUs provide much higher throughput than CPUs for dense tensor operations, yet the growth in machine learning compute demand still outpaces the growth in hardware capability. The lecture quantifies this as a gap that widens by roughly 256x every

18 months even after accelerators are taken into account [1]. The question for a deep learning system is therefore not whether to use parallel execution, but how to use it efficiently.

Memory is also a limiting resource. Large models require memory for parameters, optimizer states, gradients, activations, temporary buffers, and runtime metadata. During training, the activations produced in the forward pass are often needed later for backpropagation, so the memory footprint can be much larger than the parameter size alone. Parameter counts have grown extremely fast, on the order of 35x every two years, which is close to a doubling every 3.9 months. The progression from ResNet-50 and the original Transformer through BERT, GPT-2, Megatron-LM, GPT-3, GShard, Switch, and PaLM traces this trajectory. As model sizes grow from classic convolutional networks to modern transformer-based language models, parallelization is needed not only to obtain enough compute throughput but also to fit the training state into device memory.

3 The Training Computation as a Graph

To reason systematically about parallelization, the lecture begins with the computational graph of a training step. A training iteration contains three conceptual phases:

1. **Forward propagation.** The input batch is passed through the model to produce predictions and a loss value.
2. **Backward propagation.** Automatic differentiation traverses the graph in reverse and computes gradients of the loss with respect to model parameters and intermediate tensors.
3. **Weight update.** An optimizer such as SGD or Adam updates the parameters using the computed gradients.

For a simple two-layer example, the forward pass can be written as

$$h = \text{relu}(xW_1), \quad \hat{y} = hW_2, \quad \ell = \text{MSE}(\hat{y}, y).$$

The backward pass augments this graph with derivative operators for MSE, matrix multiplication, and relu. The update phase then adds optimizer operators, such as

$$W_t \leftarrow W_t - \eta \nabla_{W_t} \ell$$

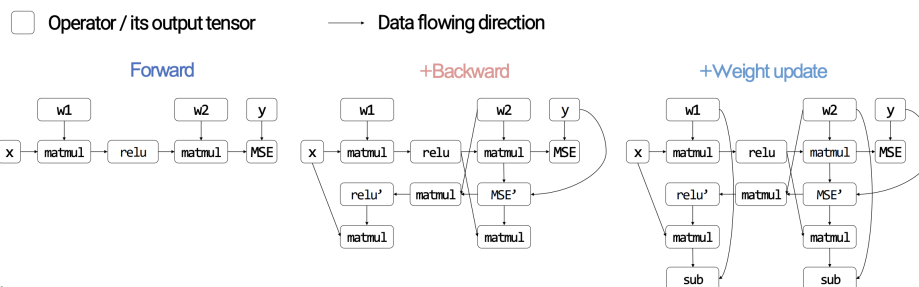
for stochastic gradient descent with learning rate η . More complex models contain many more operators, but the same graph view applies.

Recap: Computational Graph

$$\theta^{(t+1)} = f(\theta^{(t)}, \nabla_L(\theta^{(t)}, D^{(t)}))$$

$$L = \text{MSE}(w_2 \cdot \text{ReLU}(w_1 x), y) \quad \theta = \{w_1, w_2\}, D = \{(x, y)\}$$

$$f(\theta, \nabla_L) = \theta - \nabla_L$$



In this view, tensors flow along graph edges and operators sit at graph nodes. Parallelization can therefore be stated as a graph partitioning problem:

partition the training computation graph over a device cluster.

The partition must decide which device stores each tensor, which device or devices execute each operator, and what communication is needed when data crosses device boundaries.

4 Device Cluster Topology

The target hardware is not a uniform pool of identical links. A typical GPU cluster has multiple devices per node, high-bandwidth connections inside a node, and slower links between nodes. For example, GPUs inside a server may be connected through NVLink or NVSwitch, while servers communicate over network links such as InfiniBand or Ethernet. The lecture emphasizes that the topology has fast and slow connections, and a good partitioning strategy must take this heterogeneity into account.

This topology matters because graph edges become communication when their source and destination are placed on different devices. If an activation is produced on one GPU and consumed by an operator on another GPU, the runtime must transfer it. If a tensor is split across devices, the runtime may need a collective communication operation to combine partial results or redistribute shards. Thus, the cost of a parallel execution plan depends on both the computation assigned to each device and the communication induced by the assignment.

5 Classic View: Data Parallelism and Model Parallelism

The traditional taxonomy of machine learning parallelism separates data parallelism from model parallelism.

5.1 Data Parallel Training

In data parallelism, the whole model fits on each device, and different devices process different slices of the mini-batch. Suppose a mini-batch B is split across p devices:

$$B = B_1 \cup B_2 \cup \dots \cup B_p.$$

Each device i holds a replica of the parameters θ , computes a local loss $\ell_i(\theta; B_i)$, and computes local gradients

$$g_i = \nabla_{\theta} \ell_i(\theta; B_i).$$

To make the update equivalent to training on the full batch, the devices must aggregate their gradients:

$$g = \frac{1}{p} \sum_{i=1}^p g_i.$$

This aggregation is usually implemented by an *allreduce*. After the allreduce, every device has the same averaged gradient and can apply the same optimizer update.

Data parallelism is conceptually simple and works well when the model and optimizer state fit on a single device. Its main communication cost is the gradient synchronization step. In frameworks such as PyTorch, wrapping a model with distributed data parallelism (DDP) causes this gradient allreduce to happen implicitly during the backward pass.

5.2 Model Parallelism

Model parallelism is used when the computation or state of a model is split across devices. A simple form assigns different layers to different GPUs. In a transformer, for example, some blocks might be placed on one device and later blocks on another. During the forward pass, activations are sent from one stage to the next. During the backward pass, gradients flow in the opposite direction.

Pipeline model parallelism is a common layer-wise approach. The model is divided into stages, and microbatches are streamed through the pipeline so that multiple devices can be active at once. The lecture illustrates this with a small schedule across three time steps: at time 0 only the first stage is busy, at time 1 the second stage starts on the first microbatch while the first stage moves to the next microbatch, and so on. Because of dependencies between stages, some devices stay idle at the start and end of an iteration. These idle periods are the pipeline bubbles, and they motivate a more precise view of parallelism in terms of graph partitioning.

6 Computational View: Inter-op and Intra-op Parallelism

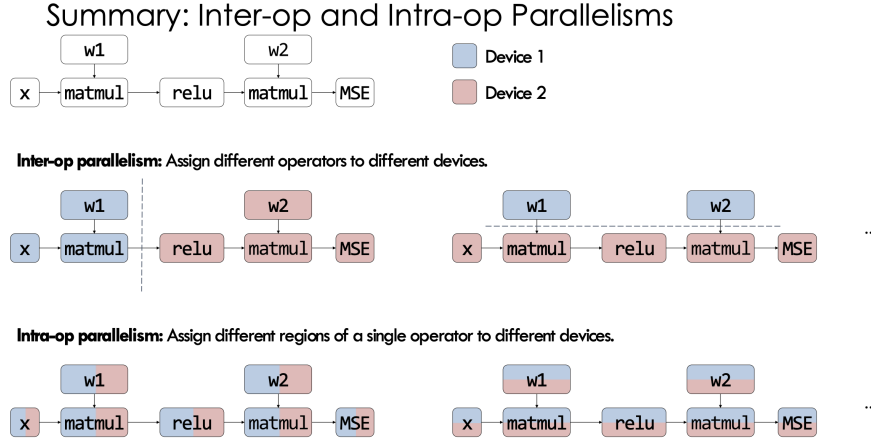
The classic terms “data parallelism” and “model parallelism” are useful, but they do not capture all possible strategies. The lecture introduces a more general computational view:

$$\text{parallelization} = \text{partitioning a computation graph on a device cluster.}$$

Under this view, the two main categories are inter-operator parallelism and intra-operator parallelism.

These two categories subsume the classic taxonomy. Data parallelism is a special case in which the model is replicated and the batch dimension is sharded, so it appears as a replication strategy whose only communication is the gradient allreduce. Model parallelism splits across devices in two distinct ways that the single

classic term conflates: assigning whole layers or stages to different devices is inter-operator parallelism, while splitting an individual operator such as a matrix multiplication across devices is intra-operator parallelism. The computational view is more general because it can describe strategies, and combinations of strategies, that the data-versus-model split cannot name cleanly.



6.1 Inter-operator Parallelism

Inter-operator parallelism assigns different operators to different devices. For example, one GPU may execute the first matrix multiplication and ReLU, while another GPU executes a later matrix multiplication and loss operator. Layer-wise model parallelism is a standard instance of inter-op parallelism.

The communication pattern is often point-to-point: an output tensor produced by one operator is sent to the device that runs the next operator. Inter-op parallelism can reduce memory pressure by spreading model state and activations across devices, but it may leave devices idle when graph dependencies are sequential: a downstream device must wait for the activation produced upstream, unless the runtime hides the gap with pipelining or by overlapping communication with computation.

6.2 Intra-operator Parallelism

Intra-operator parallelism assigns different regions of a single operator to different devices. Instead of placing an entire matrix multiplication on one GPU, the input or weight matrix can be partitioned so multiple GPUs compute partial results in parallel.

Consider a matrix multiplication

$$Y = XW,$$

where $X \in \mathbb{R}^{b \times m}$ and $W \in \mathbb{R}^{m \times n}$. There are several possible tensor parallel strategies:

- **Column partitioning.** Split the output dimension: $W = [W_1, W_2, \dots, W_p]$. Device i computes $Y_i = XW_i$. The output is sharded by columns: $Y = [Y_1, Y_2, \dots, Y_p]$. If the next operator can consume sharded output, communication may be delayed; otherwise an allgather is needed.
- **Row partitioning.** Split the input dimension: $X = [X_1, X_2, \dots, X_p]$ and $W = [W_1^T, W_2^T, \dots, W_p^T]^T$. Each device computes a partial product $Z_i = X_i W_i$, and the full output is $Y = \sum_i Z_i$. This requires a

reduction or allreduce across devices.

- **Replication.** Replicate an operator or tensor across devices. This can avoid some communication during the forward pass but typically requires synchronization later, such as gradient allreduce in data parallel training.

A particularly important pattern combines these strategies across consecutive operators. Consider the two-layer block $Y = XW_1W_2$. If W_1 is column-partitioned, each device produces a column shard of the first matrix multiplication. The elementwise ReLU between the two layers acts independently on each entry, so it can consume the sharded output directly without any communication. If W_2 is then row-partitioned to match this sharding, each device computes a partial product of the second matrix multiplication, and a single allreduce at the very end combines the partial sums into the full output. Pairing a column-partitioned layer with a following row-partitioned layer is the standard tensor-parallel construction, used for example in Megatron-style transformer layers [3], precisely because it confines communication to one collective per block rather than one after every operator.

Intra-op parallelism tends to keep devices busier because all devices can work on the same large operator at the same time. The cost is communication: partial outputs, gradients, or tensor shards often need to be exchanged using collective operations. This is why tensor-parallel implementations spend so much effort choosing partition dimensions that minimize communication and allow later operations to consume sharded tensors directly.

6.3 Trade-off

The lecture summarizes the trade-off as follows.

Parallelism style	Communication	Device idle time
Inter-operator	Less, often point-to-point	More
Intra-operator	More, often collective	Less

This table should not be interpreted as an absolute rule for every system. Rather, it captures the main tension. Inter-op strategies naturally follow the graph structure and can use simpler point-to-point transfers, but dependencies can serialize execution. Intra-op strategies expose more parallel work inside a single large operator, but they introduce expensive collectives. Modern distributed training systems often combine both styles, for example by using pipeline parallelism across groups of layers and tensor parallelism within each layer.

7 Theme Problem

The computational view leads to the central systems question of the lecture:

What is the most efficient way to execute the graph using combinations of inter-op and intra-op parallelism, subject to memory and communication constraints?

This problem is difficult because the search space is large. Each operator admits many choices (replication, row or column partitioning, layer placement, pipeline stage assignment), each tensor needs a layout, and every

edge between incompatible layouts forces communication. The best plan also depends on model shape, batch size, hardware topology, available memory, and the ability to overlap communication with computation.

A simplified version of the theme problem focuses on a single objective: maximize model FLOPs utilization subject to memory constraints. This simplification is useful because MFU is an end-to-end metric that captures whether the overall training run is using the peak compute capacity of the hardware.

8 Measuring Parallel Efficiency

8.1 Arithmetic Intensity

A local measure of efficiency is arithmetic intensity:

$$\text{AI} = \frac{\text{\#operations}}{\text{\#bytes moved}}.$$

Operations with high arithmetic intensity perform many computations per byte loaded from memory. Dense matrix multiplication is often favorable because large blocks of data can be reused many times. Element-wise operations, normalization operations, small reductions, and irregular memory accesses may have lower arithmetic intensity because they move many bytes for relatively few arithmetic operations.

Arithmetic intensity is useful for understanding individual operators, but parallel training requires a more holistic measure. Even if each operator is individually optimized, the whole program can lose time to communication, load imbalance, device idle periods, and memory stalls.

8.2 Model FLOPs Utilization

The lecture introduces model FLOPs utilization (MFU) as the macro-level metric:

$$\text{MFU} = \frac{\text{model program's total FLOPs}}{t \times \text{peak hardware FLOPs}},$$

where t is the time required to finish the program. Equivalently, MFU is the achieved model FLOP/s divided by the peak FLOP/s of the hardware.

MFU is especially important in large language model training because the hardware is extremely expensive and training runs can last for a long time. A high MFU means that the system is turning a large fraction of theoretical accelerator throughput into useful model computation. A low MFU means that the system is losing time to non-compute overheads or to inefficient compute.

The slides give concrete numbers from industry practice, pairing each reported MFU with the peak throughput that forms the denominator. On V100-class systems (about 112 TFLOPS peak), reported utilization is around 40–50%. On the A100 (about 312 TFLOPS peak), it sits near 40% before certain optimizations and rises to about 60% after optimizations such as FlashAttention. On the H100 (about 990 TFLOPS peak), it falls in the 30–50% range depending on model size. The exact number varies by model, implementation, precision, and cluster, and the next hardware generation moves the target again. The key lesson is that MFU is now a central performance metric for large-scale training systems.

8.3 Factors That Reduce MFU

Several factors can reduce MFU:

- **Operator type and shape.** Large dense matrix multiplications are usually MFU-friendly. Small operators, elementwise kernels, reductions, and memory-bound operations may underutilize the hardware.
- **Graph and operator optimization.** Kernel fusion, layout selection, memory planning, and compiler optimizations can reduce overhead and improve locality.
- **Precision and hardware generation.** The peak FLOPS depend on numerical precision and accelerator type. Tensor cores, BF16/FP16 execution, and newer GPU generations can change both the numerator and denominator in the MFU calculation.
- **Communication.** Network transfers, collectives, and synchronization can delay computation. Communication may be reduced by choosing better partitions or hidden by overlapping it with computation.
- **Load imbalance and idle time.** Some devices may receive less work, wait for dependencies, or wait at synchronization barriers.

Thus, maximizing MFU is not simply an operator-kernel problem. It requires choosing a parallelization plan that exposes enough work, minimizes unnecessary communication, respects memory limits, and schedules work so that devices stay busy.

9 Communication Primitives

Any graph partitioning plan induces communication. The lecture distinguishes between two main types of communication: point-to-point communication and collective communication.

9.1 Point-to-point Communication

Point-to-point communication transfers data from one device to another. In the computation graph view, this happens when an edge crosses a device boundary. For example, if operator (A) is placed on device 1 and operator (B) is placed on device 2, then the tensor on the edge ($A \rightarrow B$) must be sent from device 1 to device 2.

This communication is localized: it involves a producer, a consumer, and the tensor that flows between them. Point-to-point communication naturally appears in inter-operator parallelism, where different operators or layers may be assigned to different devices.

Point-to-point communication is often cheaper than a collective because it involves fewer participants and less synchronization, though it can still be expensive when it crosses a slow link or sits on the critical path.

9.2 Collective Communication

Collective communication involves a group of devices participating in the same communication operation. Instead of sending one tensor from one source to one destination, all participating devices execute a coordinated operation.

Data-parallel gradient synchronization is the canonical example. In data parallel training, each device holds a replica of the model and computes gradients on a different shard of the mini-batch. Before the optimizer update, these gradients must be synchronized so that all model replicas apply the same update. In PyTorch DDP, a training loop can look syntactically ordinary:


```
ddp_model = DDP(Model(), device_ids=[rank])
for batch in data_loader:
    loss = train_step(ddp_model, batch)
```

However, during backpropagation, DDP inserts allreduce operations so that all model replicas agree on the averaged gradients.

Collectives can be built out of many point-to-point messages, but optimized collective libraries implement them carefully for the hardware topology. Examples include NCCL for NVIDIA GPUs, RCCL for AMD GPUs, and oneCCL for Intel accelerators. These libraries choose algorithms such as rings, trees, or hierarchical schemes to reduce latency and maximize bandwidth.

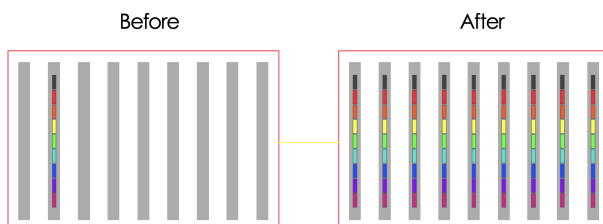
Collective operations are more global than point-to-point communication. They can move and combine data efficiently across many devices, but they also require coordination among all participants. The lecture notes that collective communication is generally not fault-tolerant: if one participant fails or stops making progress, the whole collective can stall or fail.

10 Important Collective Operations

The lecture reviews seven common collective operations. In the descriptions below, assume p devices participate and each device starts with some local buffer.

10.1 Broadcast

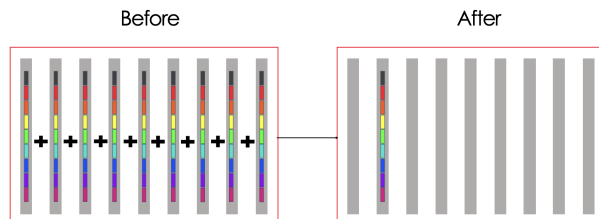
Broadcast sends data from one root device to all other devices. Before the operation, only the root holds the authoritative value. After the operation, every device holds a copy. Broadcast is useful for distributing parameters, random seeds, metadata, or any value that must become consistent across a group.



10.2 Reduce-to-one

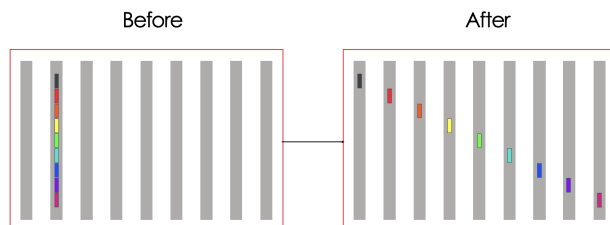
Reduce-to-one combines values from all devices and stores the result on one root device. The reduction operator is usually associative, such as sum, max, or min. For example, if each device has a scalar loss statistic, a reduce-to-one with sum can produce the global sum on the root. Unlike allreduce, only the root receives the final result.

Broadcast and reduce-to-one are dual operations. Broadcast spreads one value to many devices, while reduce-to-one gathers many contributions into one combined value.



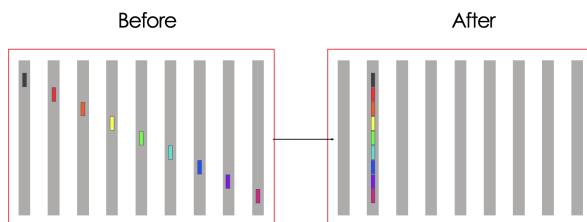
10.3 Scatter

Scatter partitions data from one root device and distributes different pieces to different devices. If the root has a tensor split into p chunks, device i receives chunk i . Scatter is useful when a single device initially owns a tensor or dataset fragment that must be shared across a group.



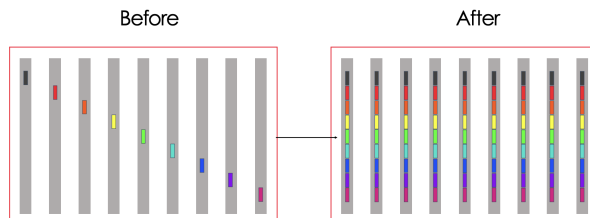
10.4 Gather

Gather is the reverse of scatter. Each device contributes a local chunk, and the root receives the concatenation or collection of all chunks. Gather is useful when sharded results must be assembled on one device. Scatter and gather are dual operations in the same sense as broadcast and reduce.



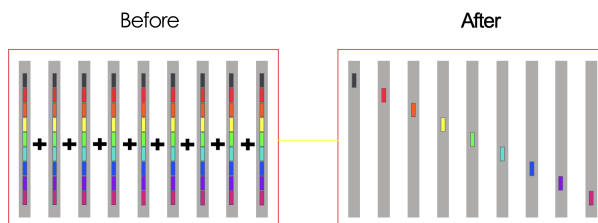
10.5 Allgather

Allgather generalizes gather by delivering the assembled result to every device, not just to a root. Each device starts with one shard; after the operation, every device has all shards. In tensor-parallel models, allgather is used when a later computation requires the full tensor on each device after earlier computations produced only sharded outputs.



10.6 Reduce-scatter

Reduce-scatter combines a reduction with a scatter. Each device contributes data, corresponding pieces are reduced across devices, and each device receives only one shard of the reduced result. Reduce-scatter is useful when the reduced output itself should remain sharded. It is often paired with allgather in distributed tensor layouts.



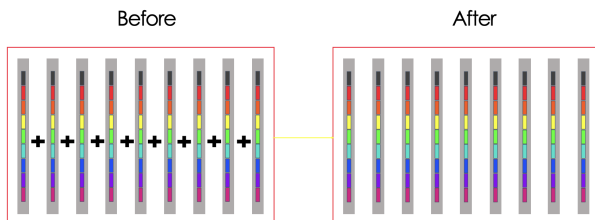
Reduce-scatter is especially useful because it performs communication while keeping the result sharded. In contrast, allreduce gives every device the full reduced tensor, which can be wasteful when each device only needs one partition of the result. For example, in distributed training with sharded optimizer states, each GPU may contribute gradients for the full parameter vector, but after reduction each GPU only needs the gradient shard corresponding to the parameters it owns. Reduce-scatter supports this pattern by summing the corresponding pieces across devices and returning only the relevant reduced shard to each device. This reduces memory usage compared with materializing the full reduced tensor everywhere.

10.7 Allreduce

Allreduce reduces values across all devices and returns the final reduced value to every device. Conceptually, allreduce is equivalent to a reduce-to-one followed by a broadcast, although optimized implementations do not necessarily execute it that way. Data-parallel gradient averaging is the standard example:

$$g_i \text{ on device } i \xrightarrow{\text{allreduce-sum}} \sum_{j=1}^p g_j \text{ on every device.}$$

The result is then divided by p if the framework wants an average rather than a sum.



Operation	Before	After
Broadcast	Root has full data	All devices have root data
Reduce-to-one	Each device has data	Root has reduced result
Scatter	Root has all shards	Each device has one shard
Gather	Each device has one shard	Root has all shards
Allgather	Each device has one shard	All devices have all shards
Reduce-scatter	Each device has data	Each device has one reduced shard
Allreduce	Each device has data	All devices have reduced result

10.8 Scribe responsibilities

- Zach Liu: Proofreading
- Lucas Yan: Refining Sections 9 and 10
- Kefan Zhang: Refining Sections 2 and 8, and adding references
- Ziang Li: Helped to draft the content
- Hongrui Zhu: Refined Section 6

References

- [1] J. Sevilla, L. Heim, A. Ho, T. Besiroglu, M. Hobbhahn, and P. Villalobos, “Compute Trends Across Three Eras of Machine Learning,” arXiv:2202.05924, 2022. <https://arxiv.org/abs/2202.05924>
- [2] T. B. Brown et al., “Language Models are Few-Shot Learners,” arXiv:2005.14165, 2020. <https://arxiv.org/abs/2005.14165>
- [3] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” arXiv:1909.08053, 2019. <https://arxiv.org/abs/1909.08053>